Face Recognition using Eigenanalysis

Eigenvalues & eigenvectors as directions and amounts of scatter

Probability Project | Computational aspect (moderate)

Rabi Shankar Sah | **Pinak Majumdar** | **Kaustubh Agtey**

Agenda

1. Problem and pipeline
2. Image data explained
3. Mean face and centering
4. Scatter: directions and amounts
5. Covariance and the size problem
6. The $A^\top A$ trick, step by step
7. A complete worked example
8. Eigenfaces and ordering
9. Choosing k (explained variance)
10. Scores and reconstruction
11. Train / test protocol
12. Recognition and results
13. Problems faced and solutions
14. Conclusion

The problem and the goal

Task: given a new face photo, decide *who* it is.

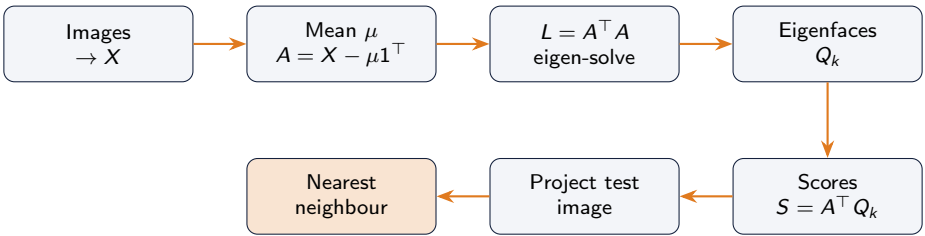
Challenges

- One photo is 36 000 numbers.
- Neighbouring pixels are highly correlated, so most numbers are redundant.
- Comparing raw pixels is slow and sensitive to noise.

Big idea

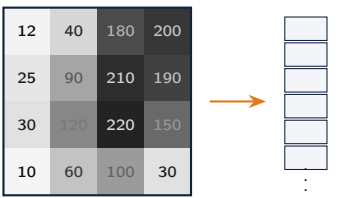
Faces vary in only a few main ways. Eigenanalysis finds those ways (directions) and tells us how much each one matters (amounts). Each face then needs only $k \ll 36000$ numbers.

Pipeline overview



Every box on this slide is derived, with numbers, on the following slides.

Image data explained: a face is a vector



grayscale grid (0 black, 255 white)

- A grayscale image is a grid of brightness values.
- Size 200×180 gives $200 \cdot 180 = 36000$ pixels.
- Read the grid row by row into one column $x \in \mathbb{R}^{36000}$.
- Every image becomes a point in a 36000-dimensional space.

The dataset and the data matrix

$$X = \begin{pmatrix} x_1 & x_2 & \cdots & x_N \end{pmatrix} \in \mathbb{R}^{36000 \times N}$$

Each column is one face, each row is one pixel position across all faces.

Notebook output

Images found: 50
People: 10 (5 images each)
Data matrix shape: (36000, 50)
Pixel range: 0–255 (kept as is, no rescaling)

Running example used on the next slides

We use a toy "face" with 3 pixels and 3 images so every number can be checked by hand:

$$x_1 = \begin{pmatrix} 3 \\ 2 \\ 3 \end{pmatrix}, \quad x_2 = \begin{pmatrix} 1 \\ 3 \\ 2 \end{pmatrix}, \quad x_3 = \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}, \qquad X = \begin{pmatrix} 3 & 1 & 2 \\ 2 & 3 & 1 \\ 3 & 2 & 1 \end{pmatrix}$$

The real data work in exactly the same way, only with 36000 rows and 50 columns.

Step 1: the mean face

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$$

Toy example, $N = 3$:

$$\mu = \frac{1}{3}\left[\begin{pmatrix}3\\2\\3\end{pmatrix} + \begin{pmatrix}1\\3\\2\end{pmatrix} + \begin{pmatrix}2\\1\\1\end{pmatrix}\right] = \frac{1}{3}\begin{pmatrix}6\\6\\6\end{pmatrix} = \begin{pmatrix}2\\2\\2\end{pmatrix}$$

In code this is the average of each row of X, where X has shape (36000, 50), giving a mean vector of shape (36000,1).



Mean face of the 50 images

Step 2: centering

$$A = X - \mu 1^\top, \qquad a_i = x_i - \mu$$

Toy example:

$$a_1 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \quad a_2 = \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}, \quad a_3 = \begin{pmatrix} 0 \\ -1 \\ -1 \end{pmatrix}, \qquad A = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Each row of A sums to 0. In the notebook this is verified: the largest absolute row mean of A is $\approx 1.1 \times 10^{-14}$ (zero up to rounding).

Consequence of centering: rank at most $N - 1$

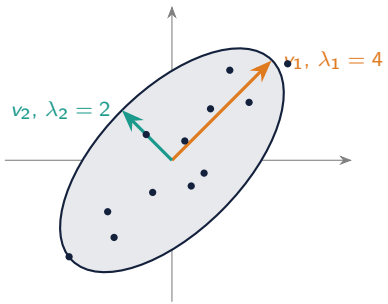
Fact

Because every row of A sums to zero, $a_1 + a_2 + \cdots + a_N = 0$. The columns are linearly dependent, so

$$\text{rank}(A) \leq N - 1.$$

- Toy: $a_1 + a_2 + a_3 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$, so rank ≤ 2.
- Real data: 50 images give at most 49 useful directions (notebook: 49 eigenfaces).
- Training set: 40 images give at most 39 (notebook: 39 valid eigenfaces).

Scatter: directions and amounts



- Centered data form a cloud around the origin.
- The cloud is stretched more in some directions than others.
- Eigenvector = a stretch direction.
- Eigenvalue = how large the spread along it is (variance).
- Longest axis = largest λ.

Eigenvalues by hand: a 2×2 example

Covariance of centered 2D data $C = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$.

Eigenvalues: $\det(C - \lambda I) = (3-\lambda)^2 - 1 = 0 \Rightarrow \lambda_1 = 4,\ \lambda_2 = 2.$

Eigenvectors:

$$\lambda_1 = 4:\ (C - 4I)v = 0 \Rightarrow v_1 = \tfrac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \lambda_2 = 2:\ v_2 = \tfrac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Check: $v_1 \cdot v_2 = 0$ (orthogonal), and $\lambda_1 + \lambda_2 = 6 = \text{trace}(C)$ (total variance).

Share of scatter along v_1: $4/6 = 66.7\%$.

Why the top eigenvector is the direction of most scatter

Variance of the data projected onto a unit direction u ($\|u\| = 1$):

$$\mathrm{Var}(u) = \frac{1}{N}\sum_i (u^\top a_i)^2 = u^\top C u.$$

Maximise $u^\top C u$ subject to $u^\top u = 1$ with a Lagrange multiplier:

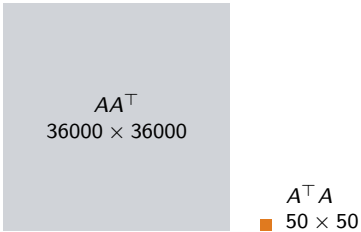
$$\mathcal{L} = u^\top C u - \lambda(u^\top u - 1), \qquad \nabla_u \mathcal{L} = 2Cu - 2\lambda u = 0 \;\Rightarrow\; Cu = \lambda u.$$

At such a point $u^\top C u = \lambda u^\top u = \lambda$. Hence the maximum variance is the largest eigenvalue, attained at its eigenvector. The next direction (orthogonal to the first) gives λ_2, and so on.

Step 3: the covariance matrix and the size problem

$$C = \frac{1}{N} A A^\top \in \mathbb{R}^{36000 \times 36000}$$

- Entries: $36000^2 = 1.296 \times 10^9$.
- Stored as 64-bit numbers: about 10.4 GB.
- Eigen-solving a matrix this large is far beyond a laptop or a Colab session.



Step 4: the $A^\top A$ trick (derivation)

Work with the small matrix

$$L = A^\top A \in \mathbb{R}^{N\times N}, \qquad Lv_i = \lambda_i v_i.$$

Multiply both sides on the left by A:

$$A(A^\top A)v_i = \lambda_i(Av_i) \quad \Longrightarrow \quad (AA^\top)(Av_i) = \lambda_i(Av_i).$$

Conclusion

Av_i is an eigenvector of the huge matrix $AA^\top$ with the same eigenvalue λ_i. We only solve a 50×50 problem. (The factor $1/N$ in C only rescales every λ_i by the same constant, so directions and ordering are unchanged.)

Worked example, part 1: build $L = A^\top A$

With the toy A and $a_1 = (1, 0, 1)$, $a_2 = (-1, 1, 0)$, $a_3 = (0, -1, -1)$, entry $L_{ij} = a_i \cdot a_j$:

$$L_{11} = 2,\ L_{22} = 2,\ L_{33} = 2, \qquad L_{12} = -1,\ L_{13} = -1,\ L_{23} = -1$$

$$L = \begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}$$

Eigenvalues: $\det(L - \lambda I) = 0$ gives $\lambda = 3,\ 3,\ 0$ (trace $= 6 = 3 + 3 + 0$).

Eigenvectors of L:

$$v_1 = \tfrac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \\ 0 \end{pmatrix}, \quad v_2 = \tfrac{1}{\sqrt{6}} \begin{pmatrix} 1 \\ 1 \\ -2 \end{pmatrix}, \quad v_3 = \tfrac{1}{\sqrt{3}} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \ (\lambda = 0)$$

Worked example, part 2: from v_i to eigenfaces

$$Av_1 = \tfrac{1}{\sqrt{2}}(a_1 - a_2) = \tfrac{1}{\sqrt{2}} \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix}, \qquad Av_2 = \tfrac{1}{\sqrt{6}}(a_1 + a_2 - 2a_3) = \tfrac{1}{\sqrt{6}} \begin{pmatrix} 0 \\ 3 \\ 3 \end{pmatrix}$$

$$Av_3 = \tfrac{1}{\sqrt{3}}(a_1 + a_2 + a_3) = 0$$

Lengths: $\|Av_1\| = \sqrt{3}$, $\|Av_2\| = \sqrt{3}$. Both equal $\sqrt{\lambda}$.

$$\boxed{u_1 = \tfrac{1}{\sqrt{6}} \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix}, \qquad u_2 = \tfrac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}}$$

Check: $u_1 \cdot u_2 = 0$ and $\|u_i\| = 1$. The eigenvalue-0 vector gives $Av_3 = 0$ (no direction) and is discarded. This is exactly the rank-$(N-1)$ fact from before.

Step 5: eigenfaces and the discard rule

$$u_i = \frac{A v_i}{\|A v_i\|}, \qquad \|A v_i\|^2 = v_i^\top L v_i = \lambda_i$$

- Normalising gives unit-length eigenfaces: $Q = [u_1 \ \cdots \ u_m]$.
- $Q^\top Q = I$ (orthonormal).
- Keep only $\lambda_i > 10^{-10}\lambda_1$ so that rounding noise is not treated as a direction.

Notebook output

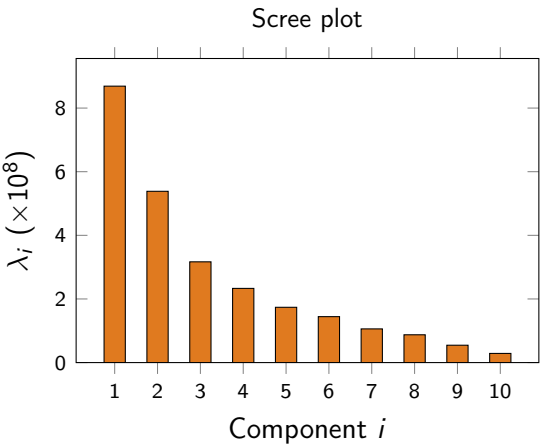
Eigenfaces: 49
Smallest eigenvalue $-2.76 \times 10^{-8} \approx 0$
(rounding noise, discarded).

Step 6: ordering the eigenvalues

Sort so that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_m > 0$.

Top values from the 50-image run:

i	λ_i
1	8.689×10^8
2	5.384×10^8
3	3.167×10^8
4	2.332×10^8
5	1.738×10^8
6	1.444×10^8
7	1.059×10^8
8	8.74×10^7
9	5.46×10^7
10	2.88×10^7



Step 7: how much variance does each direction explain?

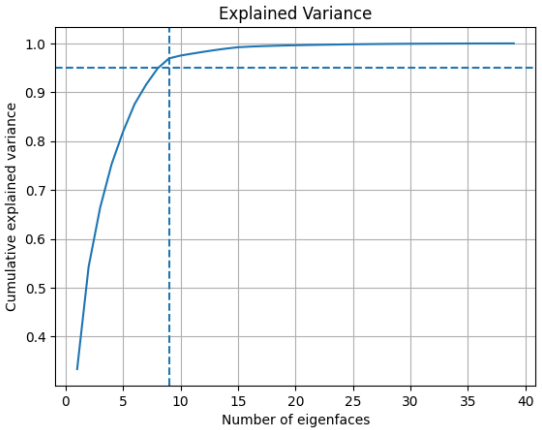
$$r_i = \frac{\lambda_i}{\sum_j \lambda_j}, \qquad R_k = \sum_{i=1}^{k} r_i, \qquad k = \min\{k : \ R_k \geq 0.95\}$$

Toy example ($\lambda = 3, 3$): $r_1 = r_2 = 0.5$, so $R_1 = 0.5$ and $R_2 = 1.0$; two directions are needed for 95%.

Notebook output (50 images)

Components for 95% variance: 9
Variance retained: 0.9568
(code: `k = searchsorted(cumulative, 0.95) + 1`; the $+1$ converts a 0-based index to a count)

Cumulative explained variance curve



- Curve rises steeply, then flattens.
- Dashed lines: 95% level and $k = 9$.
- Nine numbers carry almost all information of a 36000-number image.
- Compression: $36000/9 = 4000\times$.

Step 8: projection scores

$$S = A^\top Q_k \in \mathbb{R}^{N \times k}, \qquad s_i = Q_k^\top (x_i - \mu)$$

Toy example with $k = 1$, $u_1 = \frac{1}{\sqrt{6}}(2, -1, 1)^\top$:

$$s_1 = u_1^\top a_1 = \tfrac{3}{\sqrt{6}} \approx 1.225, \quad s_2 = u_1^\top a_2 = -\tfrac{3}{\sqrt{6}} \approx -1.225, \quad s_3 = u_1^\top a_3 = 0$$

Check: $\sum_i s_i^2 = 3 = \lambda_1$. The eigenvalue is exactly the total squared score along that direction.

Notebook shapes: scores $(50, 49)$ with all components, $(50, 9)$ with $k = 9$.

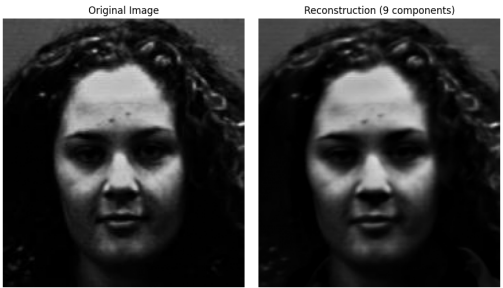
Step 9: reconstruction

$$\hat{x}_i = \mu + Q_k s_i$$

Toy, $k = 1$:

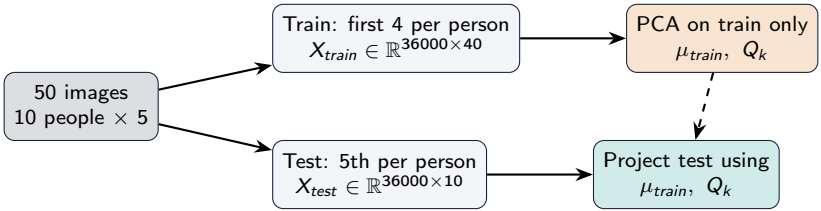
$$\hat{x}_1 = \begin{pmatrix} 2 \\ 2 \\ 2 \end{pmatrix} + \frac{3}{\sqrt{6}} \cdot \frac{1}{\sqrt{6}} \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 1.5 \\ 2.5 \end{pmatrix}$$

versus $x_1 = (3, 2, 3)^\top$: close. With $k = 2$ the result is exact, because the rank is 2.



Original vs reconstruction, 9 components

Train / test protocol (no data leakage)



- The test faces are never used to compute μ, L or Q_k.
- The test faces are treated like brand-new photos.

PCA on the training set only

Same formulas, now with $N = 40$:

$$\mu_{train} = \frac{1}{40} \sum x_i, \quad A_{train} = X_{train} - \mu_{train} 1^\top, \quad L_{train} = A_{train}^\top A_{train} \in \mathbb{R}^{40 \times 40}$$

Notebook output

Train matrix (36000, 40), test matrix (36000, 10)
Valid eigenfaces: 39 (rank $\leq N - 1$)
Selected $k = 9$ with 96.94% of the training variance
Training scores (40, 9)

Projecting a new (test) face

$$s_{test} = Q_k^\top \left(x_{test} - \mu_{train} \right) \in \mathbb{R}^k$$

- Subtract the training mean, not the test mean. The test image must be expressed in the same coordinate system as the training faces.
- Multiply by the 9 training eigenfaces to get 9 coordinates.
- Notebook: test scores have shape $(10, 9)$.

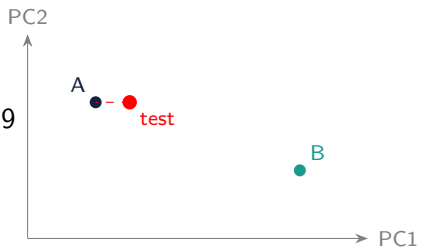
Step 10: nearest-neighbour recognition

$$d(s,t) = \sqrt{\sum_{j=1}^{k} (s_j - t_j)^2}, \qquad \hat{\imath} = \arg\min_i \|s_i^{train} - s_{test}\|, \quad \text{label} = \text{identity of image } \hat{\imath}$$

Toy example ($k = 2$): train A $= (1, 2)$,
train B $= (4, 1)$, test $= (1.5, 2)$.

$$d_A = \sqrt{0.5^2 + 0^2} = 0.50, \quad d_B = \sqrt{2.5^2 + 1^2} \approx 2.69$$

The test face is assigned to person A.



Results: 100% recognition accuracy

Test image	Actual = Predicted	Test image	Actual = Predicted
anpage	✓	pacole	✓
asamma	✓	pspliu	✓
drbost	✓	rsanti	✓
ekavaz	✓	sjbeck	✓
klclar	✓	skumar	✓

Recognition accuracy: 100.00%

10 of 10 test faces identified correctly using only 9 numbers per face.

Problems faced and solutions (1): size and memory

Problem

$C = \frac{1}{N} AA^\top$ is 36000×36000 (about 10.4 GB). Direct eigen-decomposition is not feasible.

Solution

Solve the $N \times N$ matrix $L = A^\top A$ (50×50 or 40×40) and map back with $u_i = Av_i / \|Av_i\|$. Same eigenvalues, a tiny cost.

Problems faced and solutions (2): numerical issues

Problem

Centering makes L singular. One eigenvalue is ≈ 0 and rounding makes it slightly negative (-2.76×10^{-8}). Taking $\sqrt{\lambda}$ or dividing by $\|Av\| \approx 0$ would break or blow up.

Solution

Keep only $\lambda_i > 10^{-10}\lambda_1$. This leaves 49 (full set) or 39 (training set) valid eigenfaces.

Problem

Raw eigenfaces Av_i have length $\sqrt{\lambda_i}$, so distances would be dominated by large-λ directions inconsistently.

Solution

Normalise every eigenface to unit length so that $Q^\top Q = I$ and scores are true coordinates.

Problems faced and solutions (3): method and evaluation

Problem

Computing PCA on all 50 images and then testing on the same images would leak information and overstate accuracy.

Solution

Split first (4 train / 1 test per person). Compute μ, L and Q_k from training only, and project test images with μ_{train}.

Problem

How many components? Too few lose identity, too many keep noise.

Solution

Pick the smallest k with cumulative explained variance $\geq 95\%$. Result: $k = 9$ (95.68% on 50 images, 96.94% on the training set).

Limitations and possible improvements

- Only 10 test images, so 100% is encouraging but statistically small. More people and more test photos would give a firmer estimate.
- Eigenfaces capture *overall* variance, including lighting. Strong changes in lighting, pose or expression can hurt accuracy.
- Nearest neighbour uses plain Euclidean distance. Distances weighted by $1/\sqrt{\lambda_i}$ (Mahalanobis) or a rejection threshold for unknown faces are natural extensions.
- Cross-validation (rotating which image is the test image) would give a more reliable accuracy figure.

Conclusion

- Eigenvectors of the scatter matrix give the directions in which faces vary; eigenvalues give the amount of variance in each.
- The $A^\top A$ trick turns a 36000×36000 problem into 50×50.
- Nine eigenfaces keep about 95% of the variance ($4000\times$ compression).